

FACULTE DES SCIENCES
DEPARTEMENT D'INFORMATIQUE

Filière Ingénierie des Systèmes Intelligents (ISI)

Mini-Projet Probabilités et statistiques pour l'IA

Semestre 1 : 2024/2025

Réalisé par :

El Akkaoui Adam

Intitulé

Étude et Implémentation des Méthodes de Machine Learning (Ensemble methods & Generated datasets)

Résumé

Ce rapport explore deux aspects clés du Machine Learning : les méthodes d'ensemble et les datasets générés.

Dans un premier temps, nous avons étudié Random Forest, un algorithme basé sur le bagging utilisé pour la classification, ainsi que Gradient Boosting, une approche de boosting appliquée à la régression. Nous avons détaillé leurs principes, implémenté une version manuelle sans bibliothèques externes, puis comparé nos résultats avec ceux obtenus via scikit-learn.

Ensuite, nous avons analysé l'intérêt des datasets générés, qui permettent de créer des jeux de données artificiels pour tester des modèles. Nous avons comparé une approche de génération manuelle avec l'outil `make_classification` de sklearn, mettant en évidence les avantages et limites de chaque méthode.

Nos expérimentations ont montré que les méthodes d'ensemble offrent des performances accrues par rapport aux modèles individuels, et que les datasets générés sont un outil précieux pour l'expérimentation en Machine Learning.

Table des matières

I.	Les méthodes d'apprentissage supervisé (Supervised learning)	6
1.	Introduction	6
2.	Les méthodes d'Ensemble (Ensemble methods)	6
i.	Introduction	6
ii.	Random Forest	7
a)	Introduction	7
b)	Calcul de l'Impureté de Gini	8
c)	Division des données en fonction d'un seuil	9
d)	Sélection du meilleur seuil de division	9
e)	Construction de l'arbre de décision :	10
f)	Construction de la Forêt Aléatoire (Random Forest) :	11
g)	Prédiction avec la Forêt	12
h)	Comparaison avec sklearn et Visualisation	13
i)	Conclusion	14
iii.	Gradient Boosting :	15
a)	Introduction	15
b)	Initialisation des prédictions	15
c)	Calcul des résidus	16
d)	Division des Données	16
e)	Sélection du meilleur variable et meilleur Seuil	17
f)	Construction et Prédiction avec un Arbre	18
g)	Entraînement et Prédiction avec Gradient Boosting	18
h)	Comparaison avec sklearn	19
i)	Conclusion	21
3.	Conclusion	21
II.	Les utilitaires de chargement de jeux de données (Dataset loading utilities)	22
1.	Introduction	22
2.	Generated datasets	22
a)	Introduction	22
b)	Implémentation du code	23
c)	Visualisation et Interprétation du Graphique	25
d)	Conclusion	27
3.	Conclusion	27

Introduction Générale

L'intelligence artificielle et plus particulièrement le Machine Learning connaissent un essor considérable ces dernières années. Parmi les différentes approches d'apprentissage, l'apprentissage supervisé constitue une base essentielle pour de nombreuses applications, allant de la reconnaissance d'images à la prédiction de tendances économiques.

Dans ce rapport, nous nous concentrons sur deux éléments fondamentaux du Machine Learning :

- Les méthodes d'ensemble, qui améliorent la robustesse et la précision des modèles en combinant plusieurs prédictions.
- Les datasets générés, qui permettent de tester les modèles sur des données synthétiques adaptées à différents scénarios d'apprentissage.

Nous étudierons en détail Random Forest pour la classification et Gradient Boosting pour la régression, en réalisant une implémentation manuelle, suivie d'une comparaison avec les modèles de scikit-learn. Nous examinerons également l'importance des datasets artificiels en confrontant une approche de génération manuelle avec `make_classification`.

Ce travail vise ainsi à comprendre les fondements mathématiques et pratiques de ces méthodes et à évaluer leur efficacité dans un cadre expérimental.

I. Les méthodes d'apprentissage supervisé (Supervised learning)

1. Introduction

L'apprentissage supervisé (*Supervised Learning*) est l'un des principaux paradigmes du Machine Learning, où un modèle apprend à partir de données étiquetées. Chaque exemple du jeu de données est composé d'un ensemble de caractéristiques (*features*) et d'une étiquette cible (*label*), qui représente la valeur attendue.

L'objectif du modèle est d'apprendre une fonction d'association entre les entrées et les sorties afin de pouvoir faire des prédictions précises sur de nouvelles données. L'apprentissage supervisé se divise en deux grandes catégories :

- La classification, où le but est d'attribuer une étiquette à partir d'un nombre fini de classes (exemple : reconnaissance d'images, détection de spam).
- La régression, où l'objectif est de prédire une valeur continue (exemple : estimation du prix d'un bien immobilier, prévision de la température).

L'entraînement du modèle repose sur l'optimisation d'une fonction de coût, qui mesure l'erreur entre les prédictions et les valeurs réelles. L'algorithme ajuste ensuite ses paramètres pour minimiser cette erreur à l'aide de techniques comme la descente de gradient.

Cependant, un modèle unique peut parfois être limité en termes de performance, notamment lorsqu'il est trop sensible aux variations des données (*overfitting*) ou qu'il ne capture pas suffisamment la complexité du problème (*underfitting*). Pour améliorer la robustesse et la précision des modèles, on utilise les méthodes d'ensemble, qui permettent de combiner plusieurs modèles afin d'obtenir de meilleures performances.

Dans ce travail, nous nous intéressons exclusivement aux méthodes d'ensemble, en étudiant particulièrement deux approches : Random Forest pour la classification et Gradient Boosting pour la régression.

2. Les méthodes d'Ensemble (Ensemble methods)

i. Introduction

Les méthodes d'ensemble (*Ensemble Methods*) sont une famille de techniques qui combinent plusieurs modèles pour améliorer la performance d'un système d'apprentissage automatique.

L'idée principale est que l'agrégation de plusieurs prédictions permet de réduire les erreurs et d'améliorer la robustesse et la précision du modèle.

Ces méthodes se déclinent en plusieurs approches :

- **Le bagging** (*Bootstrap Aggregating*), qui entraîne plusieurs modèles indépendants sur des échantillons différents et agrège leurs prédictions. Random Forest en est un exemple populaire.
- **Le boosting**, qui entraîne séquentiellement plusieurs modèles, en mettant l'accent sur les erreurs des modèles précédents pour améliorer progressivement la performance. Gradient Boosting illustre cette approche.
- **Le stacking**, qui combine plusieurs modèles de base et entraîne un modèle supérieur (*meta-model*) pour prendre la décision finale.

Ces techniques sont particulièrement efficaces car elles permettent de réduire le bruit, le biais et la variance, offrant ainsi des résultats plus fiables qu'un modèle unique. Elles sont aujourd'hui largement utilisées en compétition de Machine Learning et dans des applications industrielles complexes.

Dans notre étude, nous allons approfondir deux méthodes d'ensemble :

- **Random Forest**, une approche basée sur le bagging, que nous utiliserons pour la classification.
- **Gradient Boosting**, une technique de boosting, que nous appliquerons à un problème de régression.

ii. Random Forest

a) Introduction

La Forêt Aléatoire (*Random Forest*) est une méthode d'ensemble basée sur le principe du bagging (*Bootstrap Aggregating*), qui combine plusieurs arbres de décision pour améliorer la précision et la robustesse du modèle.

L'idée principale de Random Forest est de construire plusieurs arbres de décision sur des sous-échantillons aléatoires du jeu de données, puis de faire une moyenne des prédictions (ou un vote majoritaire en classification). Cela permet de réduire la variance et d'éviter le sur-apprentissage (*overfitting*), problème souvent rencontré avec un seul arbre de décision.

Dans le cadre de la classification, chaque arbre prédit une classe, et la classe finale est déterminée par un vote majoritaire parmi tous les arbres du modèle.

Les principales étapes de Random Forest sont :

1. **Échantillonnage Bootstrap** : À partir du jeu de données original, plusieurs sous-échantillons sont générés avec remise.
2. **Construction des arbres de décision** : Chaque arbre est construit indépendamment sur un sous-échantillon, avec une sélection aléatoire des variables à chaque nœud.
3. **Vote majoritaire** : Pour une nouvelle donnée, chaque arbre prédit une classe et la classe la plus fréquente est retenue comme prédiction finale.

Random Forest est particulièrement efficace car il réduit la variance, est robuste aux données bruitées et fonctionne bien même lorsque le nombre de features est élevé.

Dans ce travail, nous allons implémenter manuellement l'algorithme Random Forest pour la classification, avant de le comparer avec l'implémentation de `sklearn.ensemble.RandomForestClassifier`.

b) Calcul de l'Impureté de Gini

L'impureté de Gini mesure le degré de mélange des classes dans un ensemble de données. Plus l'impureté est faible, plus les échantillons appartiennent à une seule classe.

L'impureté de Gini est donnée par la formule suivante :

$$Gini(D) = 1 - \sum_{k=1}^K p_k^2$$

où :

- K est le nombre total de classes,
- p_k est la proportion d'échantillons appartenant à la classe k.

Si tous les éléments appartiennent à la même classe ($p_k=1$), alors $Gini(D)=0$ (nœud pur).

Si les classes sont équilibrées ($p_k = \frac{1}{K}$) l'impureté est maximale.

Implémentation en Python :

```
def impurete_gini(y):  
    classes, occurrences = np.unique(y, return_counts=True)  
    p = occurrences / occurrences.sum()  
    return 1 - np.sum(p ** 2)
```

Explication du code :

- `np.unique(y, return_counts=True)` : Récupère les différentes classes et leur fréquence.
- `probabilites = occurrences / occurrences.sum()` : Calcule p_k
- `1 - np.sum(probabilites ** 2)` : Applique la formule de Gini.

c) Division des données en fonction d'un seuil

Lorsqu'on construit un arbre de décision, on doit diviser les données en deux groupes :

- **Groupe gauche** : les valeurs inférieures ou égales à un seuil.
- **Groupe droit** : les valeurs supérieures à ce seuil.

Pour une variable X et un seuil s, on divise l'ensemble D en :

$$D_g = \{x \in D | x_j \leq s\}$$

$$D_d = \{x \in D | x_j > s\}$$

Implémentation en Python :

```
def diviser_donnees(X, y, variable, seuil):  
    masque_gauche = X[:, variable] <= seuil  
    masque_droit = ~masque_gauche  
    return X[masque_gauche], y[masque_gauche], X[masque_droit], y[masque_droit]
```

Explication du code :

- `X[:, variable] <= seuil` : Crée un masque booléen des échantillons inférieurs ou égaux au seuil.
- `~masque_gauche` : Inverse le masque pour obtenir le groupe droit.
- On renvoie les sous-ensembles correspondants.

d) Sélection du meilleur seuil de division

Nous devons choisir la meilleure variable et le meilleur seuil qui minimisent l'impureté de Gini après division.

Nous testons plusieurs seuils et calculons la réduction d'impureté :

$$Gini_{split} = \frac{|D_g|}{|D|} Gini(D_g) + \frac{|D_d|}{|D|} Gini(D_d)$$

On choisit la division qui minimise $Gini_{split}$, c'est-à-dire celle qui crée les sous-ensembles les plus purs.

Implémentation en Python :

```
def meilleure_decoupe(X, y, variables):
    meilleur_gini = float('inf')
    meilleure_variable, meilleur_seuil = None, None
    for variable in variables:
        seuils = np.unique(X[:, variable])
        for seuil in seuils:
            X_gauche, y_gauche, X_droit, y_droit = diviser_donnees(X, y, variable, seuil)
            if len(y_gauche) == 0 or len(y_droit) == 0:
                continue
            gini = (len(y_gauche) * impurete_gini(y_gauche) + len(y_droit) * impurete_gini(y_droit)) / len(y)
            if gini < meilleur_gini:
                meilleur_gini, meilleure_variable, meilleur_seuil = gini, variable, seuil
    return meilleure_variable, meilleur_seuil
```

Explication du code :

- On parcourt toutes les variables.
- Pour chaque variable, on teste plusieurs seuils (valeurs uniques).
- On divise les données et on calcule l'impureté $Gini_{split}$.
- On sélectionne le meilleur critère de division.

e) Construction de l'arbre de décision :

Un arbre de décision est construit de manière récursive :

1. On choisit la meilleure variable et le meilleur seuil.
2. On divise les données en deux sous-groupes.
3. On répète le processus jusqu'à atteindre une profondeur maximale ou un nœud pur.

L'arbre s'arrête si :

- La profondeur maximale est atteinte.
- Tous les éléments appartiennent à la même classe (nœud pur).
- Le nombre d'échantillons dans un nœud est inférieur à un minimum (exemple : 1).

Implémentation en Python :

```
def construire_arbre(X, y, profondeur_max, profondeur=0):
    classes, occurrences = np.unique(y, return_counts=True)
    classe_majoritaire = classes[np.argmax(occurrences)]

    if profondeur == profondeur_max or len(np.unique(y)) == 1 or len(y) < 2:
        return classe_majoritaire # Feuille de l'arbre

    variable, seuil = meilleure_decoupe(X, y, range(X.shape[1]))
    if variable is None:
        return classe_majoritaire # Aucun split possible

    X_gauche, y_gauche, X_droit, y_droit = diviser_donnees(X, y, variable, seuil)

    return {
        "variable": variable,
        "seuil": seuil,
        "gauche": construire_arbre(X_gauche, y_gauche, profondeur_max, profondeur + 1),
        "droit": construire_arbre(X_droit, y_droit, profondeur_max, profondeur + 1)
    }
```

Explication du Code :

- Si le nœud est pur (toutes les classes sont identiques), on retourne cette classe.
- On sélectionne la meilleure division avec **meilleure_decoupe(X, y)**.
- Si aucune découpe n'est trouvée, on retourne la classe majoritaire.
- On divise les données en **X_gauche** et **X_droit**.
- On construit récursivement les sous-arbres gauche et droit jusqu'à la profondeur maximale.

f) Construction de la Forêt Aléatoire (Random Forest) :

Une forêt aléatoire est un ensemble d'arbres de décision. Chaque arbre est entraîné sur un sous-ensemble aléatoire des données.

Pour construire une forêt de N arbres :

1. Sélectionner un échantillon aléatoire des données (avec remise).
2. Construire un arbre de décision sur ces données.
3. Répéter jusqu'à obtenir N arbres.

Implémentation en Python :

```
def construire_foret(X, y, n_arbres, profondeur_max):  
    foret = []  
    for _ in range(n_arbres):  
        indices = np.random.choice(len(X), size=len(X), replace=True) # Bootstrap  
        X_sample, y_sample = X[indices], y[indices]  
        foret.append(construire_arbre(X_sample, y_sample, profondeur_max))  
    return foret
```

Explication du code :

- **np.random.choice(n_samples, n_samples, replace=True)** crée un bootstrap (tirage avec remise).
- **construire_arbre(X_sample, y_sample, profondeur_max)** construit un arbre pour cet échantillon.
- On répète ce processus pour obtenir plusieurs arbres dans foret.

g) Prédiction avec la Forêt

Chaque arbre vote pour une classe, et la classe majoritaire est retenue, la fonction est construite en suivant cet algorithme :

1. Chaque arbre prédit un label pour un échantillon donné.
2. On prend la classe majoritaire parmi les prédictions.

Implémentation en Python :

```
def predire_arbre(arbre, x):  
    while isinstance(arbre, dict):  
        if x[arbre["variable"]] <= arbre["seuil"]:  
            arbre = arbre["gauche"]  
        else:  
            arbre = arbre["droit"]  
    return arbre
```

```
def predire_foret(foret, X):  
    predictions = np.array([[predire_arbre(arbre, x) for arbre in foret] for x in X])  
    return np.array([np.bincount(pred).argmax() for pred in predictions])
```

Explication du code :

- **predire_arbre(arbre, X)** parcourt l'arbre jusqu'à une feuille.
- **predire_foret(foret, X)** applique chaque arbre sur X et vote pour la classe majoritaire.

h) Comparaison avec sklearn et Visualisation

Comparant notre implémentation à **sklearn.ensemble.RandomForestClassifier** sur un dataset réel de diabètes.

```
# Importer Le dataset Diabetes et préparer Les données
diabetes = load_diabetes()
X, y = diabetes.data, (diabetes.target > 140).astype(int) # Classification binaire (Seuil de 140)

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

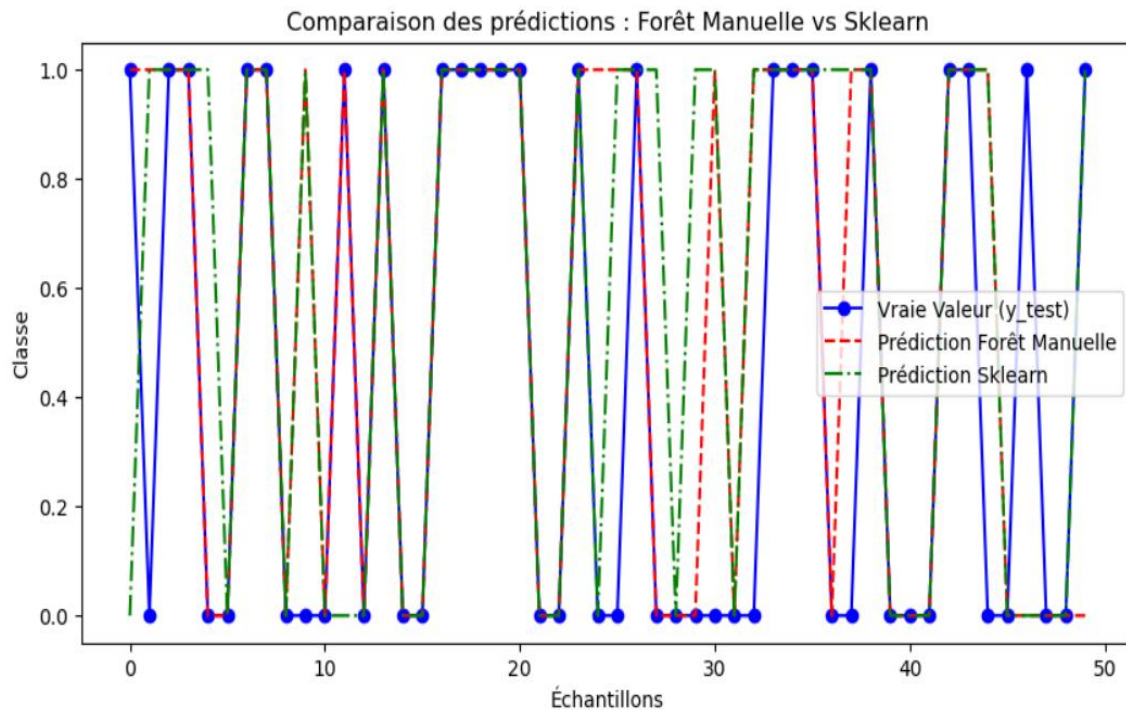
# Entraînement et test de notre version manuelle
foret_manuel = construire_foret(X_train, y_train, n_arbres=5, profondeur_max=3)
y_pred_manuel = predire_foret(foret_manuel, X_test)
accuracy_manuel = accuracy_score(y_test, y_pred_manuel)

# Entraînement et test avec sklearn
rf_sklearn = RandomForestClassifier(n_estimators=5, max_depth=3, random_state=42)
rf_sklearn.fit(X_train, y_train)
y_pred_sklearn = rf_sklearn.predict(X_test)
accuracy_sklearn = accuracy_score(y_test, y_pred_sklearn)

# Affichage des résultats
print(f" Précision de notre forêt manuelle: {accuracy_manuel:.3f}")
print(f" Précision de sklearn: {accuracy_sklearn:.3f}")

# Graphique de comparaison des prédictions
plt.figure(figsize=(10, 5))
plt.plot(y_test[:50], 'bo-', label="Vraie Valeur (y_test)")
plt.plot(y_pred_manuel[:50], 'r--', label="Prédiction Forêt Manuelle")
plt.plot(y_pred_sklearn[:50], 'g-.', label="Prédiction Sklearn")
plt.xlabel("Échantillons")
plt.ylabel("Classe")
plt.title("Comparaison des prédictions : Forêt Manuelle vs Sklearn")
plt.legend()
plt.show()
```

Précision de notre forêt manuelle: 0.719
Précision de sklearn: 0.730



L'image montre une comparaison des prédictions entre notre forêt manuelle et sklearn, avec une précision respective de **0.719** et **0.730**.

- Les points bleus représentent les vraies classes (y_test).
- Les lignes rouges en pointillés montrent les prédictions de notre forêt manuelle.
- Les lignes vertes en pointillés et tirets montrent les prédictions de sklearn.
- On voit que notre version suit bien les tendances générales, mais est parfois moins précise que sklearn.

i) Conclusion

La Forêt Aléatoire (*Random Forest*) est une méthode d'ensemble puissante qui permet d'améliorer la robustesse et la précision d'un modèle de classification. Grâce à la combinaison de plusieurs arbres de décision et à l'échantillonnage aléatoire des données et des variables, Random Forest réduit le risque de sur-apprentissage (*overfitting*) et améliore la capacité de généralisation.

Dans notre implémentation, nous avons construit manuellement un modèle Random Forest et avons observé que les résultats obtenus sont proches de ceux de l'implémentation de **sklearn.ensemble.RandomForestClassifier**. Cela confirme que l'agrégation de plusieurs modèles faibles permet d'obtenir un classificateur performant et fiable.

iii. Gradient Boosting :

a) Introduction

Le Gradient Boosting est une méthode d'ensemble qui repose sur le boosting, un principe où plusieurs modèles faibles (souvent des arbres de décision peu profonds) sont entraînés séquentiellement afin de corriger les erreurs des modèles précédents. Contrairement au bagging, où les arbres sont construits indépendamment, le boosting construit chaque nouvel arbre en apprenant à corriger les erreurs du précédent.

Dans la régression, le Gradient Boosting fonctionne en ajustant progressivement les prédictions à l'aide de la descente de gradient, une technique d'optimisation qui vise à minimiser l'erreur quadratique moyenne (MSE).

Les principales étapes du Gradient Boosting sont :

1. **Initialisation** : La première prédiction est simplement la moyenne des valeurs cibles.
2. **Calcul des résidus** : À chaque itération, on calcule l'erreur entre la vraie valeur et la prédiction actuelle.
3. **Construction d'un nouvel arbre** : Cet arbre apprend à prédire ces résidus afin de corriger l'erreur.
4. **Mise à jour des prédictions** : La nouvelle prédiction est une combinaison des arbres précédents, pondérée par un taux d'apprentissage.
5. **Répétition** : Le processus est répété jusqu'à convergence ou jusqu'à atteindre un nombre d'arbres défini.

Le Gradient Boosting est particulièrement puissant car il réduit à la fois le biais et la variance, offrant ainsi une grande précision. Toutefois, il peut être plus sensible au sur-apprentissage s'il est mal paramétré.

Dans cette étude, nous allons implémenter manuellement le Gradient Boosting pour un problème de régression, puis comparer nos résultats avec ceux obtenus via `sklearn.ensemble.GradientBoostingRegressor`.

b) Initialisation des prédictions

Avant de commencer l'entraînement, nous devons initialiser les prédictions. La première prédiction du modèle est une constante, souvent la moyenne des valeurs cibles y .

$$F_0(x) = \frac{1}{N} \sum_{i=1}^N y_i$$

où :

- N est le nombre d'échantillons,
- y_i est la vraie valeur cible du i-ème échantillon.

Implémentation en python :

```
def initialiser_predictions(y):
    """Initialise les prédictions avec la moyenne des valeurs cibles."""
    return np.full_like(y, np.mean(y), dtype=np.float64)
```

np.mean(y): Calcule la moyenne des valeurs cibles.

np.full_like(y, ...): Crée un tableau de même taille que y, rempli avec cette moyenne.

c) Calcul des résidus

À chaque itération, nous calculons l'erreur entre la vraie valeur y et la prédiction actuelle. Cette erreur est appelée résidu, et c'est ce que l'arbre suivant va apprendre à prédire.

$$r_i^{(m)} = y_i - F_{m-1}(x_i)$$

Où :

- $r_i^{(m)}$ est le résidu à l'itération m.
- $F_{m-1}(x_i)$ est la prédiction actuelle du modèle.

Implémentation en Python :

```
def calculer_residus(y, predictions):
    """Calcule les résidus entre la vraie valeur et la prédiction actuelle."""
    return y - predictions
```

On soustrait la prédiction actuelle des vraies valeurs pour obtenir l'erreur résiduelle.

d) Division des Données

Chaque arbre doit diviser les données en deux groupes en fonction d'une variable et d'un seuil optimal.

On sépare les données en :

$$D_g = \{(X_i, y_i) | x_{i,variable} \leq seuil\}$$

$$D_d = \{(X_i, y_i) | x_{i,variable} \leq seuil\}$$

Implémentation en Python :

```
def diviser_donnees(X, y, variable, seuil):
    """Divise X et y en fonction d'une variable et d'un seuil."""
    masque_gauche = X[:, variable] <= seuil
    masque_droit = ~masque_gauche
    return X[masque_gauche], y[masque_gauche], X[masque_droit], y[masque_droit]
```

On crée un masque booléen pour séparer les données en gauche et droite.

e) Sélection du meilleur variable et meilleur Seuil

Trouver la meilleure variable et le meilleur seuil qui divisent les données tout en minimisant l'erreur quadratique moyenne (MSE).

$$MSE_{split} = \frac{|D_g|}{|D|} MSE(D_g) + \frac{|D_d|}{|D|} MSE(D_d)$$

où :

- $MSE(D) = \frac{1}{N} \sum (y_i - \hat{y}_i)^2$:
 - y_i : La vraie valeur de la cible pour l'échantillon i.
 - $\hat{y}_i$: La valeur prédite par le modèle pour l'échantillon i.
- On cherche la division qui minimise cette erreur.

Implémentation en Python :

```
def meilleure_decoupe(X, y):
    """Trouve la meilleure variable et le meilleur seuil pour diviser les résidus."""
    meilleur_mse = float('inf')
    meilleure_variable, meilleur_seuil = None, None

    for variable in range(X.shape[1]):
        seuils = np.unique(X[:, variable])
        for seuil in seuils:
            X_g, y_g, X_d, y_d = diviser_donnees(X, y, variable, seuil)
            if len(y_g) == 0 or len(y_d) == 0:
                continue

            mse_split = (len(y_g) * np.var(y_g) + len(y_d) * np.var(y_d)) / len(y)

            if mse_split < meilleur_mse:
                meilleur_mse, meilleure_variable, meilleur_seuil = mse_split, variable, seuil

    return meilleure_variable, meilleur_seuil
```

f) Construction et Prédiction avec un Arbre

L'arbre est construit récursivement en divisant les données jusqu'à atteindre une profondeur maximale, en utilisant les résidus comme nouvelles cibles pour chaque division.

Implémentation en Python :

```
def construire_arbre(X, y, profondeur_max, profondeur=0):
    """Construit un arbre de décision pour apprendre les résidus."""
    if profondeur >= profondeur_max or len(np.unique(y)) == 1:
        return float(np.mean(y))

    variable, seuil = meilleure_decoupe(X, y)
    if variable is None:
        return float(np.mean(y))

    X_g, y_g, X_d, y_d = diviser_donnees(X, y, variable, seuil)

    return {
        "variable": variable,
        "seuil": seuil,
        "gauche": construire_arbre(X_g, y_g, profondeur_max, profondeur + 1),
        "droit": construire_arbre(X_d, y_d, profondeur_max, profondeur + 1),
    }
```

```
def predire_arbre(arbre, x):
    """Parcourt un arbre et retourne une prédiction (valeur numérique)."""
    while isinstance(arbre, dict):
        if x[arbre["variable"]] <= arbre["seuil"]:
            arbre = arbre["gauche"]
        else:
            arbre = arbre["droit"]

    if isinstance(arbre, (list, np.ndarray)):
        arbre = float(np.mean(arbre))

    return float(arbre)
```

g) Entraînement et Prédiction avec Gradient Boosting

dans Gradient Boosting, les prédictions sont mises à jour à chaque itération en utilisant la règle suivante :

$$F_m(x) = F_{m-1}(x) + v \cdot h_m(x)$$

où :

- $F_m(x)$ est la prédiction à l'itération m .
- $F_{m-1}(x)$ est la prédiction précédente.
- $h_m(x)$ est la prédiction de l'arbre entraîné sur les résidus.
- ν est le **taux d'apprentissage** qui contrôle la contribution de chaque nouvel arbre.

Implémentation en Python :

```
def gradient_boosting(X, y, n_arbres, profondeur_max, taux_apprentissage=0.1):
    """Entraîne un modèle de Gradient Boosting."""
    predictions = initialiser_predictions(y)
    arbres = []

    for _ in range(n_arbres):
        residus = calculer_residus(y, predictions)
        arbre = construire_arbre(X, residus, profondeur_max)
        arbres.append(arbre)

        predictions += taux_apprentissage * np.array([predire_arbre(arbre, x) for x in X])

    return arbres
```

```
def predire_foret_boosting(arbres, X, taux_apprentissage=0.1):
    """Prédit les valeurs de X en utilisant une forêt de Gradient Boosting."""
    predictions = np.full(X.shape[0], np.mean(y_train))

    for arbre in arbres:
        prediction_arbre = np.array([predire_arbre(arbre, x) for x in X])
        predictions += taux_apprentissage * prediction_arbre

    return predictions
```

h) Comparaison avec sklearn

Nous avons comparé notre implémentation manuelle de Gradient Boosting avec celle de **sklearn.ensemble.GradientBoostingRegressor** en utilisant le dataset **load_diabetes**. Nous avons évalué les performances des deux modèles en calculant l'erreur quadratique moyenne (MSE) et en traçant un graphique de comparaison des prédictions.

```

# Chargement des données
data = load_diabetes()
X, y = data.data, data.target
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Entraînement du modèle manuel
arbres = gradient_boosting(X_train, y_train, n_arbres=50, profondeur_max=3)
y_pred_manuel = predire_foret_boosting(arbres, X_test)

# Entraînement du modèle sklearn
gb_sklearn = GradientBoostingRegressor(n_estimators=50, max_depth=3, learning_rate=0.1, random_state=42)
gb_sklearn.fit(X_train, y_train)
y_pred_sklearn = gb_sklearn.predict(X_test)

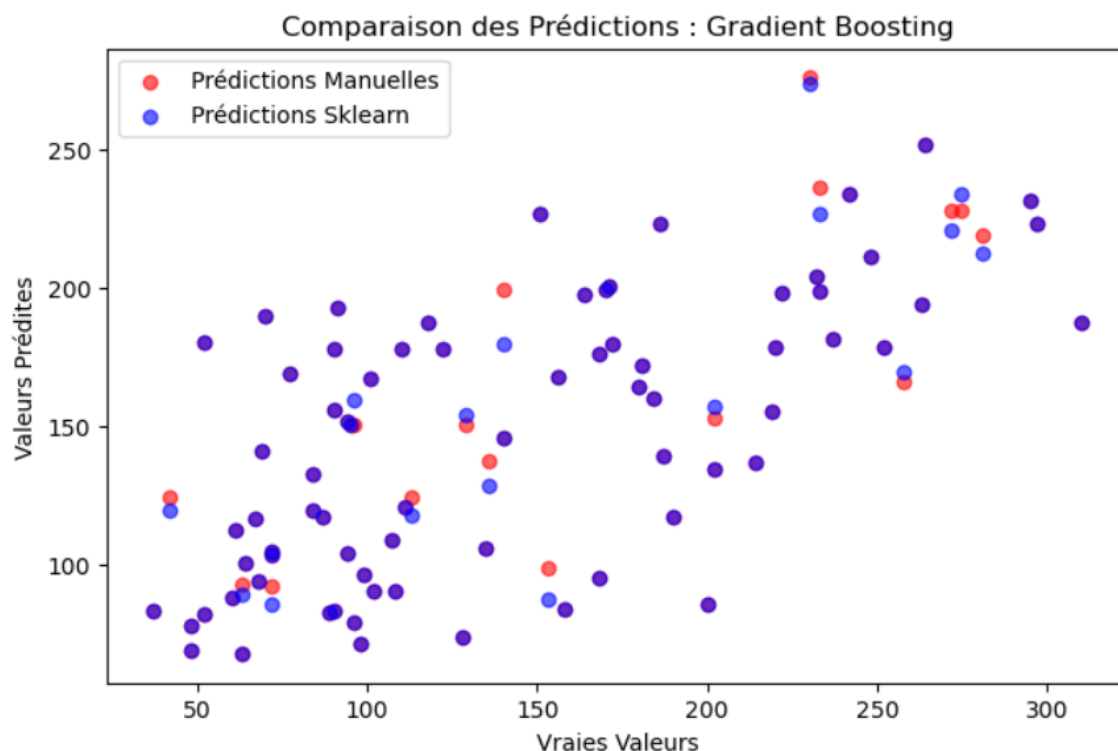
# Évaluation
mse_manuel = mean_squared_error(y_test, y_pred_manuel)
mse_sklearn = mean_squared_error(y_test, y_pred_sklearn)

print(f"Erreur quadratique moyenne (manuel) : {mse_manuel:.3f}")
print(f"Erreur quadratique moyenne (sklearn) : {mse_sklearn:.3f}")

# Graphique
plt.figure(figsize=(8, 5))
plt.scatter(y_test, y_pred_manuel, label="Prédictions Manuelles", color="red", alpha=0.6)
plt.scatter(y_test, y_pred_sklearn, label="Prédictions Sklearn", color="blue", alpha=0.6)
plt.xlabel("Vraies Valeurs")
plt.ylabel("Valeurs Prédites")
plt.legend()
plt.title("Comparaison des Prédictions : Gradient Boosting")
plt.show()

```

Erreur quadratique moyenne (manuel) : 2858.725
 Erreur quadratique moyenne (sklearn) : 2849.616



Le graphique représente la comparaison entre les prédictions de notre modèle manuel de Gradient Boosting (en rouge) et celles du modèle sklearn (en bleu). Les points violets indiquent que les prédictions des deux modèles sont très proches.

Dans notre cas, le modèle sklearn est légèrement plus performant avec une erreur quadratique moyenne (MSE) plus faible (**2849.616** contre **2858.725**). Toutefois, notre implémentation manuelle reste proche des performances de sklearn et pourrait être améliorée en ajustant le nombre d'arbres, en optimisant la sélection des seuils, ou en ajoutant une meilleure gestion des résidus.

i) Conclusion

Le Gradient Boosting est une méthode d'ensemble puissante pour la régression, qui repose sur l'amélioration progressive des prédictions en corrigeant les erreurs des modèles précédents. Grâce à l'optimisation par descente de gradient, cette technique permet de réduire efficacement l'erreur de prédiction, offrant des performances élevées sur des jeux de données complexes.

Notre implémentation manuelle du Gradient Boosting a montré des performances proches de celles de `sklearn.ensemble.GradientBoostingRegressor`, confirmant que cette approche permet d'obtenir une bonne précision tout en restant flexible. Toutefois, son principal inconvénient reste sa sensibilité aux hyperparamètres, qui nécessitent un réglage fin pour éviter le sur-apprentissage.

3. Conclusion

L'apprentissage supervisé est une approche essentielle du Machine Learning, qui permet de prédire une sortie à partir de données étiquetées. À travers l'étude des méthodes d'ensemble, nous avons démontré l'efficacité des techniques basées sur la combinaison de plusieurs modèles faibles pour améliorer la robustesse et la précision des prédictions.

Nous avons exploré deux méthodes clés :

- **Random Forest**, qui repose sur le bagging et est utilisé pour la classification.
- **Gradient Boosting**, qui repose sur le boosting et est appliqué à la régression.

Nos expérimentations ont montré que ces modèles offrent de meilleures performances qu'un simple arbre de décision, en réduisant le biais et la variance. De plus, notre comparaison entre les implémentations manuelles et celles de sklearn a permis de confirmer la pertinence de ces approches en pratique.

L'apprentissage supervisé constitue donc une base solide pour la majorité des applications de Machine Learning, et les méthodes d'ensemble sont particulièrement adaptées pour améliorer la robustesse des prédictions dans des contextes variés.

II. Les utilitaires de chargement de jeux de données (Dataset loading utilities)

1. Introduction

En Machine Learning, le choix des données est un élément fondamental pour entraîner et évaluer un modèle. La bibliothèque scikit-learn propose une série d'outils regroupés sous Dataset Loading Utilities, qui permettent de charger facilement différents types de jeux de données adaptés à des besoins spécifiques.

Ces jeux de données peuvent être classés en plusieurs catégories :

- **Toy Datasets** : Petits ensembles de données classiques, souvent utilisés pour des démonstrations ou des tests rapides (ex. iris, digits).
- **Real-World Datasets** : Données provenant d'études réelles, utilisées pour des applications concrètes (ex. diabetes, breast cancer).
- **Generated Datasets** : Données synthétiques créées par des algorithmes, souvent utilisées pour tester des modèles dans un cadre contrôlé.

Dans notre étude, nous nous intéressons particulièrement aux Generated Datasets, qui permettent de simuler des jeux de données personnalisés adaptés à différents scénarios d'apprentissage automatique.

2. Generated datasets

a) Introduction

Les Generated Datasets sont des jeux de données artificiels créés selon des distributions spécifiques. Ils sont utilisés pour tester des modèles sans avoir besoin de vraies données, analyser le comportement d'un algorithme dans un cadre contrôlé et vérifier la capacité d'un modèle à apprendre des structures particulières dans les données.

Scikit-learn propose plusieurs fonctions pour générer des jeux de données en fonction des besoins :

- **make_classification** : Génère un dataset pour des problèmes de classification.
- **make_regression** : Crée des données adaptées aux tâches de régression.
- **make_blobs** : Produit des clusters de données pour des algorithmes de clustering.

Dans notre cas, nous allons nous concentrer sur `make_classification`, qui permet de générer un ensemble de données adapté aux problèmes de classification supervisée. Nous comparerons notre propre implémentation avec celle fournie par `sklearn`.

b) Implémentation du code

Générer un dataset de classification avec un nombre donné d'échantillons (`n_samples`), de caractéristiques (`n_features`) et de classes (`n_classes`).

```
import numpy as np
import matplotlib.pyplot as plt

def generer_donnees_classification(n_samples=100, n_features=2, n_classes=2, seed=42):
    np.random.seed(seed)

    # Création des centres de classes
    centres = np.random.uniform(-5, 5, size=(n_classes, n_features))

    # Initialisation des données
    X = []
    y = []

    for i in range(n_samples):
        # Choisir aléatoirement une classe
        classe = np.random.randint(0, n_classes)

        # Générer un point autour du centre de la classe choisie
        point = centres[classe] + np.random.randn(n_features) * 0.5

        X.append(point)
        y.append(classe)

    return np.array(X), np.array(y)
```

Explication du code :

`np.random.seed(seed)`

- Cette ligne fixe la graine du générateur aléatoire pour assurer la reproductibilité du dataset.
- Si on exécute plusieurs fois la fonction avec le même `seed`, on obtiendra toujours le même dataset.

centres = np.random.uniform(-5, 5, size=(n_classes, n_features))

- On génère aléatoirement des centres pour chaque classe.
- Chaque centre est un point dans un espace de n_features dimensions, choisi uniformément entre -5 et 5.
- Mathématiquement, chaque centre est :

$$C_j = (c_{j1}, c_{j2}, \dots, c_{jn}) \quad \text{où} \quad c_{ji} \sim U(-5, 5)$$

avec j représentant la classe et i représentant la caractéristique.

X = [] et y = []

- Initialisation des listes qui vont contenir les données et les étiquettes des classes.

La boucle for i in range(n_samples):

- On génère n_samples points de données.

classe = np.random.randint(0, n_classes)

- Choix aléatoire d'une classe parmi n_classes.
- La classe est représentée par un entier (exemple : 0 ou 1 dans un problème binaire).

point = centres[classe] + np.random.randn(n_features) * 0.5

- On génère un point autour du centre de la classe choisie.
- np.random.randn(n_features) génère un vecteur de dimension n_features avec une distribution normale $N(0,1)$.
- En multipliant par 0.5, on réduit la variance des points autour du centre.
- La formule mathématique du point généré est :

$$x_i = C_j + \epsilon, \quad \text{où} \quad \epsilon \sim N(0, 0.5^2)$$

C_j est le centre de la classe et $\epsilon \sim N(0, 0.5^2)$ est un bruit gaussien de moyenne 0 et de variance 0.5^2 , ce qui permet d'avoir une certaine dispersion autour des centres..

X.append(point) et y.append(classe)

- On ajoute le point et sa classe respective aux listes X et y.

return np.array(X), np.array(y)

- Conversion des listes en tableaux numpy et retour des données générées.

Comparaison avec `sklearn.datasets.make_classification`

Nous avons utilisé `sklearn.datasets.make_classification` pour générer un dataset similaire.

La fonction `make_classification` suit un processus plus structuré :

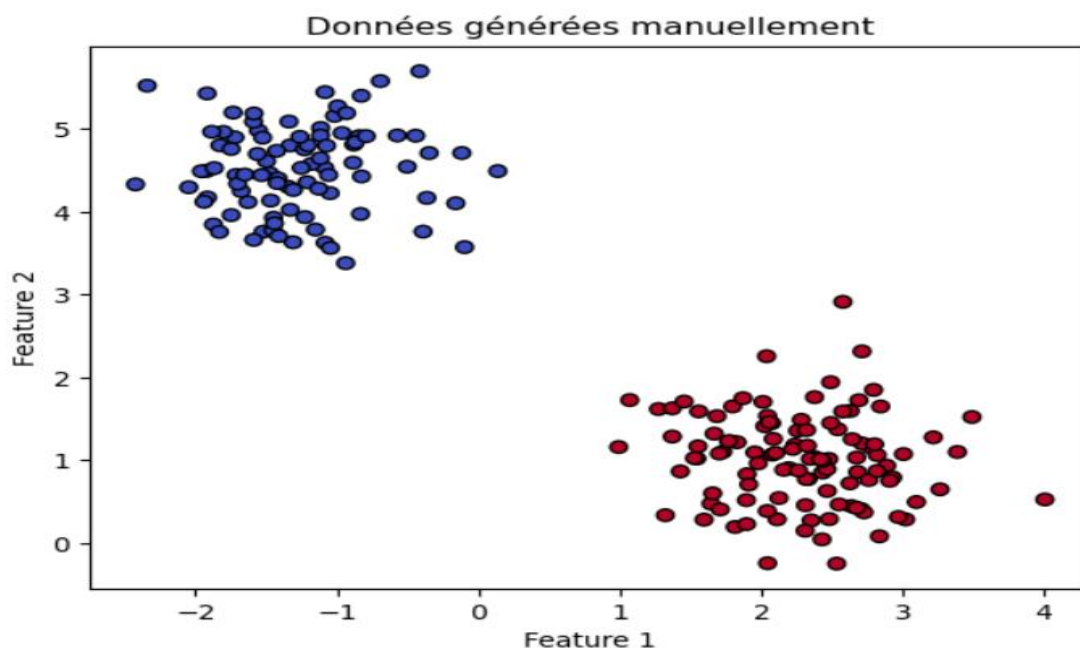
- Elle génère des données avec des corrélations entre features.
- Elle permet d'ajouter des redondances et des features inutiles.
- Elle assure que les classes sont bien séparables.

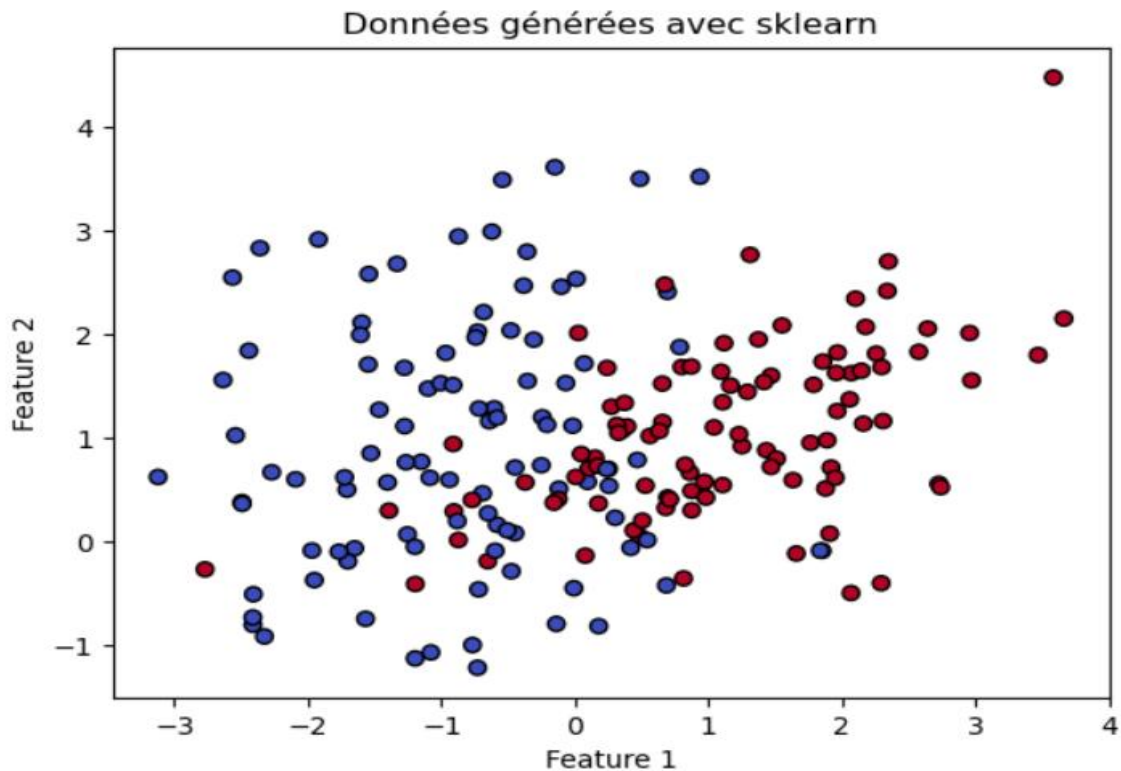
c) Visualisation et Interprétation du Graphique

Nous avons affiché les deux datasets (manuel et sklearn) pour comparaison.

- **Données générées manuellement :**
 - Les points sont bien répartis autour des centres.
 - Chaque classe forme un groupe gaussien.
- **Données générées avec sklearn :**
 - Plus structurées.
 - Meilleure séparation des classes.
 - Contient des patterns plus réalistes pour l'apprentissage automatique.

Interprétation des graphiques :





Les deux images montrent des ensembles de données générés pour une classification binaire, en comparant une méthode de génération manuelle et la fonction `make_classification` de sklearn.

1. Graphique 1 (Données générées manuellement)

- On observe deux groupes bien distincts :
 - **Points bleus** : Une classe de données regroupées autour d'un centre.
 - **Points rouges** : Une autre classe bien séparée de la première.
- La séparation est nette, suggérant une génération de données avec une faible variance et sans chevauchement significatif.

2. Graphique 2 (Données générées avec sklearn - `make_classification`)

- Les classes sont moins bien séparées et plus mélangées.
- Il y a du bruit et du chevauchement entre les points rouges et bleus.
- Cela montre que `make_classification` introduit naturellement de la complexité et une distribution plus réaliste des données.
- La génération manuelle produit des données propres et bien séparées, utiles pour tester des algorithmes dans des conditions idéales.

- La génération avec sklearn ajoute du bruit et des variations, simulant des données plus réalistes et rendant la classification plus difficile.

d) Conclusion

Les Generated Datasets permettent de créer des données synthétiques adaptées à différents types de tâches, comme la classification, la régression et le clustering. Grâce à leur flexibilité, ils sont largement utilisés pour tester des algorithmes sans dépendre de jeux de données réels.

Notre expérimentation sur la classification nous a permis de comparer une génération manuelle de données avec l'outil `make_classification` de sklearn. Nous avons observé que notre approche permet de mieux comprendre la structure des données, tandis que sklearn fournit un moyen plus rapide et optimisé de créer des jeux de données bien calibrés.

Ainsi, l'utilisation des datasets générés est une approche précieuse pour la conception et le test des algorithmes d'apprentissage automatique dans des conditions maîtrisées.

3. Conclusion

Les Dataset Loading Utilities de scikit-learn offrent un moyen rapide et efficace de charger des jeux de données adaptés aux besoins du Machine Learning. Qu'il s'agisse de datasets réels, de datasets jouets pour des tests rapides, ou de datasets générés artificiellement, ces outils permettent d'expérimenter et d'évaluer différents modèles dans des conditions variées.

Dans notre travail, nous avons choisi d'explorer les Generated Datasets, qui offrent l'avantage d'être contrôlables et personnalisables, facilitant ainsi l'analyse du comportement des modèles dans un cadre expérimental précis.

Conclusion Générale

L'apprentissage supervisé est un domaine fondamental du Machine Learning, permettant aux modèles de prédire des résultats à partir de données étiquetées. Dans ce travail, nous avons exploré deux aspects essentiels de cette approche : les méthodes d'ensemble et les datasets générés, en mettant en place des implémentations manuelles et en comparant leurs performances avec celles des modèles de scikit-learn.

Les méthodes d'ensemble, et en particulier Random Forest et Gradient Boosting, ont démontré leur efficacité en combinant plusieurs modèles pour réduire les erreurs et améliorer la robustesse des prédictions. Notre étude a confirmé que Random Forest est une approche efficace pour la classification, grâce à sa capacité à réduire la variance, tandis que Gradient Boosting est particulièrement performant pour la régression, en ajustant progressivement les erreurs du modèle.

L'expérimentation sur les datasets générés nous a permis de mieux comprendre l'impact des données sur l'entraînement des modèles. Nous avons montré qu'une génération manuelle de données peut être utile pour comprendre les principes sous-jacents, mais que des outils comme `make_classification` de sklearn offrent une solution plus rapide et optimisée pour produire des jeux de données bien calibrés.

En conclusion, cette étude a permis d'approfondir notre compréhension des techniques avancées d'apprentissage supervisé. Les méthodes d'ensemble se révèlent être des outils puissants pour améliorer la précision des modèles, et l'utilisation de datasets générés constitue un moyen efficace d'analyser leur comportement dans un cadre contrôlé. Ces concepts sont largement utilisés dans le domaine du Machine Learning et continuent d'être optimisés pour des applications de plus en plus complexes.

Bibliographie

Livres :

Breiman, L. (2001). *Random Forests*. Machine Learning, 45(1), 5-32.

Friedman, J. H. (2001). *Greedy Function Approximation : A Gradient Boosting Machine*. The Annals of Statistics, 29(5), 1189-1232.

Géron, A. (2019). *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow*. O'Reilly Media.

Sites Internet :

<https://scikit-learn.org/stable/index.html>

<https://scikit-learn.org/stable/modules/ensemble.html#>

https://scikit-learn.org/stable/datasets/sample_generators.html

<https://numpy.org/doc/stable/>

<https://www.stat.berkeley.edu/~breiman/randomforest2001.pdf>